

Module 6

Complex Multi-Agent & Protocol Testbenches (SV/UVM)

Learning objectives

- Apply your planning, environment, and regression skills to **complex multi-agent, protocol-heavy te...

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Multi-Agent Environment Desi"]

T2["Protocol Agents and Checkers"]

T1 --> T2

T3["Scoreboards and Integration"]

T2 --> T3

Apply your planning, environment, and regression skills to **complex multi-agent, protocol-heavy testbenches** (e.g., AXI-like, multi-chann...

Overview

- By Module 6, you have:

Design architecture

1. Multi-agent / multi-channel TB

- Multiple agents (per interface or channel) coordinated by env and virtual sequences.
- Layered architecture: block agents → fabric/scoreboard → subsystem scenarios
(`MULTI_AGENT_ARCHITEC...

Rtl Architecture

```
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
    subgraph DUT["stream_fifo (common_dut/rtl)"]
        SRC["Source interface\ns_valid / s_ready / s_data"]
        MEM["Circular buffer\nmem[DEPTH], wr_ptr, rd_ptr"]
        SNK["Sink interface\nm_valid / m_ready / m_data"]
        STS["Status\nlevel, overflow, underflow"]
    end
```

2. Protocol verification layer

- Dedicated protocol checker (SVA or monitor-based) separate from scoreboard data checks.
- `PROTOCOL\_VERIFICATION\_PLAN.md` lists rules, coverage, and integration with agents.

Verification Architecture

```
Diagram: Verification Architecture
(render with mmdc when available)
flowchart TB
  subgraph SYS["Multi-agent / protocol TB"]
    A1["Agent: channel A"]
    A2["Agent: channel B"]
    PC["Protocol checker"]
    SB["System scoreboard"]
```

3. System integration view

- `INTEGRATION\_PLAN.md` describes how block-level env connects to subsystem tests.
- Reuse agents/VIP across DUT variants via configuration and factory overrides.

Monitor observes transactions

```
class stream_monitor extends uvm_component;

    `uvm_component_utils(stream_monitor)

    virtual stream_if.slave vif;
    uvm_analysis_port #(stream_item) ap;

    function new(string name, uvm_component parent);
        super.new(name, parent);
        ap = new("ap", this);
    endfunction

    task run_phase(uvm_phase phase);
        stream_item tr;

        if (vif == null) begin
            # ...
```

Multi-agent architecture template

```
# Multi-Agent & Multi-Channel Architecture (Module 6)
```

```
> **Purpose**: Describe the architecture and coordination of multi-agent, multi-channel UVM environments for your DUT.
```

```
> **Module**: 6 – Complex Multi-Agent & Protocol Testbenches (SV/UVM)
```

```
> **Instructions**: Fill in this document for your design. Copy from `module6/templates/` if needed, or edit the file in `module6/` directly.
```

```
Reference: `module6/.solutions/`.
```

```
## 1. Context
```

```
<!-- TODO: DUT/System, interfaces/channels, existing env. Why a multi-agent design is required. -->
```

```
## 2. High-Level Environment Diagram
```

Verification & testing methods

1. Protocol rule testing

- Each protocol rule maps to checker fires, coverpoints, and at least one directed or random test.
- Illegal sequences and corner timing documented with expected checker behavior.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

RULES["Protocol rules"] --> CHK["Checker / monitor"]

CHK --> COV["Protocol coverage"]

INT["Integration scenarios"] --> SYS["Multi-agent stimulus"]

SYS --> CHK

INT --> SB["End-to-end scoreboard"]

2. Multi-agent scenario testing

- Virtual sequences stress cross-agent ordering, arbitration, and backpressure.
- System scoreboard validates end-to-end data and sideband signals.

3. Maintainability reviews

- Architecture review before adding agents — avoid duplicate monitors and conflicting checkers.
- ``./scripts/module6.sh --check`` validates multi-agent and protocol planning artifacts.

Syllabus topics

1. Multi-Agent Environment Design

- Multi-agent and multi-channel environment design.
- Layered and reusable testbench architecture patterns.

2. Protocol Agents and Checkers

- Protocol agent and protocol checker design.

3. Scoreboards and Integration

- Integration of complex scoreboards and time-based matching.

4. Debug and Analysis

- Debugging and analyzing complex, protocol-heavy simulations.

Hands-on examples

Module 6 self-check

```
$ ./scripts/module6.sh --help
```

Terminal: module6_check

```
[[0;36m===== [[0m
[[0;36mModule 6: Complex Multi-Agent & Protocol Testbenches [[0m
[[0;36m===== [[0m
```

Usage: ./scripts/module6.sh [OPTIONS]

Module 6: Complex Multi-Agent & Protocol Testbenches (SV/UVM)
This script summarizes and checks the planning artifacts for Module 6.

OPTIONS:

```
--multi-agent-status  Show status of multi_agent_architecture.md only
--protocol-status     Show status of protocol_verification_plan.md only
--integration-status  Show status of integration_plan.md only
--checklist-status    Show status of checklist_module6.md only
--summary             Show all statuses (default)
--check               Media/CI self-check (structure only)
--demo               Print example commands from EXAMPLES.md
--scaffold            Copy templates/*.md into module6/ if missing
--help, -h           Show this help message
```

EXAMPLES:

```
# Full summary
./scripts/module6.sh
```

```
# Focus only on multi-agent architecture
./scripts/module6.sh --multi-agent-status
```

Exercise scaffold

```
./scripts/module6.sh --scaffold
```

Demo: Protocol verification plan

```
$ head -30 module6/templates/PROTOCOL_VERIFICATION_PLAN.md
```

Terminal: ex01_protocol_plan

```
# Protocol Verification Plan (Module 6)
```

```
> **Purpose**: Define how you will verify protocol correctness (e.g., AXI-like, custom bus) usin  
> **Module**: 6 – Complex Multi-Agent & Protocol Testbenches (SV/UVM)
```

```
> **Instructions**: Fill in this document for your design. Copy from `module6/templates/` if nee
```

```
## 1. Protocol Overview
```

```
<!-- TODO: Protocol name, key characteristics (channels, handshakes, ordering, error/response se
```

```
## 2. Protocol Agent Strategy
```

```
<!-- TODO: Table of Agent Name, Role (master/slave/passive), Channels Handled, Notes. Active vs
```

```
## 3. Protocol Rules and Checkers
```

```
### 3.1 Rule Inventory
```

```
<!-- TODO: Table of Rule ID, Description, Type (safety/order/timing), Related Req IDs, Notes. --
```

```
### 3.2 Checker Design
```

```
<!-- TODO: Table of Checker ID, Location, Inputs, Rules Enforced. Placement and error reporting.
```

```
## 4. Protocol Coverage
```

```
### 4.1 Functional Coverage for Protocol
```

```
<!-- TODO: Coverage IDs, purpose, placement/sampling. Link to module3/COVERAGE_DESIGN.md. -->
```

Demo: Multi-agent architecture

```
$ head -25 module6/templates/MULTI_AGENT_ARCHITECTURE.md
```

Terminal: ex02_multi_agent

Multi-Agent & Multi-Channel Architecture (Module 6)

> **Purpose**: Describe the architecture and coordination of multi-agent, multi-channel UVM env
> **Module**: 6 - Complex Multi-Agent & Protocol Testbenches (SV/UVM)

> **Instructions**: Fill in this document for your design. Copy from `module6/templates/` if nee

1. Context

<!-- TODO: DUT/System, interfaces/channels, existing env. Why a multi-agent design is required.

2. High-Level Environment Diagram

<!-- TODO: Describe multi-agent env hierarchy (uvm_test_top, env, agents). Annotate agents, inst

3. Agent Inventory and Roles

<!-- TODO: Table of Agent Name, Role (master/slave/passive), Interface/Channel(s), Active/Passiv

4. Coordination Strategy

<!-- TODO: Virtual sequencers, scenarios. How agents are coordinated (sequences, backpressure, c

5. Multi-Channel Scoreboarding

Demo: Integration plan

```
$ head -25 module6/templates/INTEGRATION_PLAN.md
```

Terminal: ex03_integration

Integration & System-Level Scenarios (Module 6)

> **Purpose**: Describe how your components, agents, and protocols integrate at block, subsystem
> **Module**: 6 – Complex Multi-Agent & Protocol Testbenches (SV/UVM)

> **Instructions**: Fill in this document for your design. Copy from `module6/templates/` if nee

1. Integration Levels

<!-- TODO: Table of Level (block/subsystem/system), Description, Scope/Components Included. Whic

2. Integration Architecture

<!-- TODO: How agents and components connect at chosen integration level. Agents, cross-block, r

3. System-Level Scenarios

<!-- TODO: Table of Scenario ID, Description, Level, Agents/Interfaces Involved, Notes. Precondi

4. Cross-Block / Cross-Agent Checking

<!-- TODO: How end-to-end correctness is verified across components. Combined scoreboard, cross-

5. Configuration Across Levels

Demo: Reference protocol plan

```
$ head -20 module6/.solutions/PROTOCOL_VERIFICATION_PLAN.md
```

Terminal: ex04_solutions

```
# Protocol Verification Plan (Module 6)
```

```
> **Purpose**: Define how you will verify protocol correctness (e.g., AXI-like, custom bus) using  
> **Module**: 6 – Complex Multi-Agent & Protocol Testbenches (SV/UVM)
```

```
## 1. Protocol Overview
```

- ****Protocol Name****: Ready/Valid streaming (source and sink).
- ****Key Characteristics****:
 - ****Channels****: Source (s_valid, s_ready, s_data); sink (m_valid, m_ready, m_data).
 - ****Handshakes****: Transfer on clock edge when valid & ready; one beat per handshake.
 - ****Ordering****: FIFO order — first-in first-out.
 - ****Error/response semantics****: Overflow (write when full) and underflow (read when empty) set

Summarize how the protocol is used by the DUT (roles, typical traffic patterns):

- ****DUT roles****: stream_fifo is sink on source interface (accepts push) and source on sink interface
- ****Traffic****: Single-beat transfers; backpressure via ready deassertion; typical patterns = sus

```
## 2. Protocol Agent Strategy
```

Demo: Module validation

```
$ ./scripts/module6.sh --check
```

Terminal: ex05_validate

```
[[0;36m===== [[0m
[[0;36mModule 6: Complex Multi-Agent & Protocol Testbenches [[0m
[[0;36m===== [[0m
```

```
Module 6: media self-check
OK: module6/ exists
OK: templates/ exists
OK: EXAMPLES.md exists
OK: docs/MODULE6.md exists
OK: common_dut/rtl/ exists
```

All required checks passed.

Practice & assessment

What you should know (1/2)

- Design and document multi-agent UVM environments for protocol-based DUTs.
- Implement and integrate protocol agents and protocol checkers.
- Apply layered architecture patterns to keep complex benches maintainable.
- Analyze and debug multi-channel, protocol-level behavior in simulations.
- Multi-agent/multi-channel environment design documented in ``module6/MULTI_AGENT_ARCHITECTURE.md``.
- Protocol verification strategy documented in ``module6/PROTOCOL_VERIFICATION_PLAN.md``.

What you should know (2/2)

- Integration patterns and system-level scenarios documented in `module6/INTEGRATION\_PLAN.md`.
- Complex bench design reviewed; action items captured in the docs.

Assessment checklist

- Multi-agent/multi-channel environment design
documented in `module6/MULTI\_AGENT\_ARCHITECTURE.md`.
- Protocol verification strategy documented in
`module6/PROTOCOL\_VERIFICATION\_PLAN.md`.
- Integration patterns and system-level scenarios
documented in `module6/INTEGRATION\_PLAN.md`.
- Complex bench design reviewed; action items captured
in the docs.

Summary & next steps

- Apply your planning, environment, and regression skills to ****complex multi-agent, protocol-heavy te...**
- Complete module6/CHECKLIST.md
- Review module6/EXAMPLES.md and run each lab
- Next: Next module in course